

AUTENTICADOR DE VOZ

Alumno

Aguilar Felix Jesus Antonio
Cesar Enrique Vazquez Cardenas
Espinoza Mendivil Jorge Alberto
Medina Robert Alexander

Materia

Inteligencia Artificial

Horario

7AM - 8AM

Maestro

Jose Luis
Medina Jimenez

TECNOLOGICO DE CULIACAN
INGIENERIA EN SISTEMAS
COMPUTACIONALES

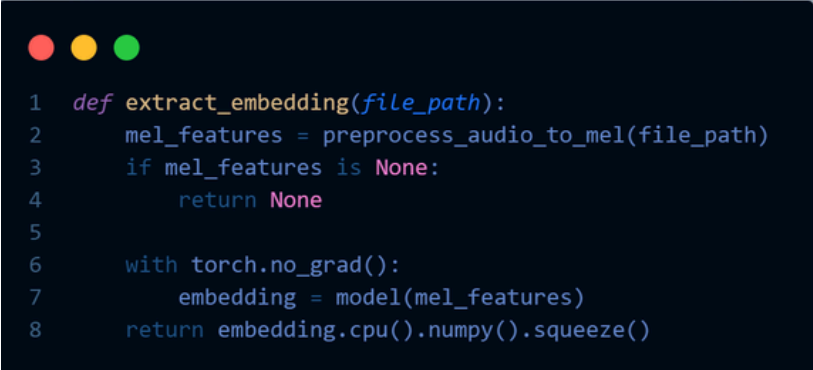
Introducción

El objetivo de este proyecto es implementar un sistema capaz de identificar o autenticar la voz de una persona utilizando técnicas de Deep Learning. El núcleo del proyecto es el uso de "embeddings" generados por el modelo VoiceEmbedder, el cual está basado en Whisper.

El sistema funciona convirtiendo una muestra de voz en un vector numérico (embedding) que representa sus características únicas. Posteriormente, un modelo de Machine Learning (como SVM o MLP) se entrena para clasificar estos vectores y asociarlos con una persona específica.

El proyecto se dividió en tres fases principales, implementadas en tres scripts de Python.

1. Carga y Preprocesamiento: Cada archivo de audio se carga usando librosa.
2. Extracción de Características: Se utiliza el WhisperProcessor para convertir el audio crudo en un espectrograma Mel.
3. Generación del Embedding: El espectrograma Mel se pasa al modelo VoiceEmbedder, que lo transforma en un vector (embedding) de dimensionalidad fija.
4. Almacenamiento: Todos los vectores (embeddings) se guardan en un archivo embeddings.npy, y sus etiquetas correspondientes (los nombres de las personas) se guardan en labels.npy.



```
1 def extract_embedding(file_path):
2     mel_features = preprocess_audio_to_mel(file_path)
3     if mel_features is None:
4         return None
5
6     with torch.no_grad():
7         embedding = model(mel_features)
8     return embedding.cpu().numpy().squeeze()
```

Entrenamiento y Evaluación (2_entrenar_evaluar.py)

Una vez que tenemos la base de datos de embeddings, esta fase se encarga de entrenar y evaluar los modelos de clasificación.

1. Carga de Datos: Se leen los archivos embeddings.npy (datos, X) y labels.npy (etiquetas, y).
2. Codificación: Las etiquetas de texto (nombres) se convierten a números usando LabelEncoder de Scikit-learn.
3. División: Los datos se dividen en conjuntos de entrenamiento y prueba (train_test_split).
4. Entrenamiento: Se inicializan y entrenan los clasificadores solicitados (SVM y/o MLP) con los datos de entrenamiento.

- Guardado: Los modelos entrenados y el LabelEncoder se guardan en archivos .joblib para su uso en la aplicación en vivo.
- Evaluación: Se generan las métricas de rendimiento (Accuracy, F1-Score) y la matriz de confusión usando el conjunto de prueba.

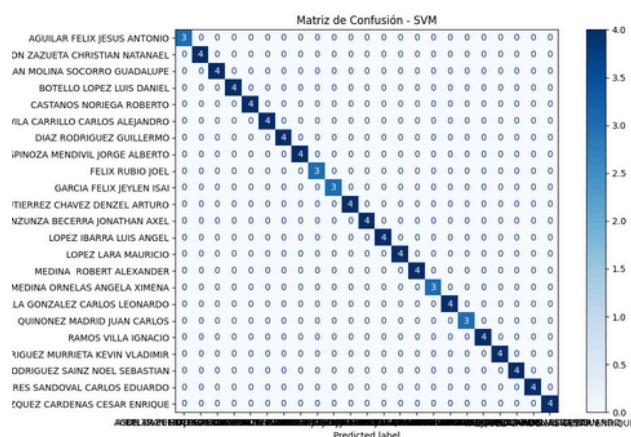
```
1 svm_model = SVC(kernel='rbf', probability=True, C=10, random_state=42)
```

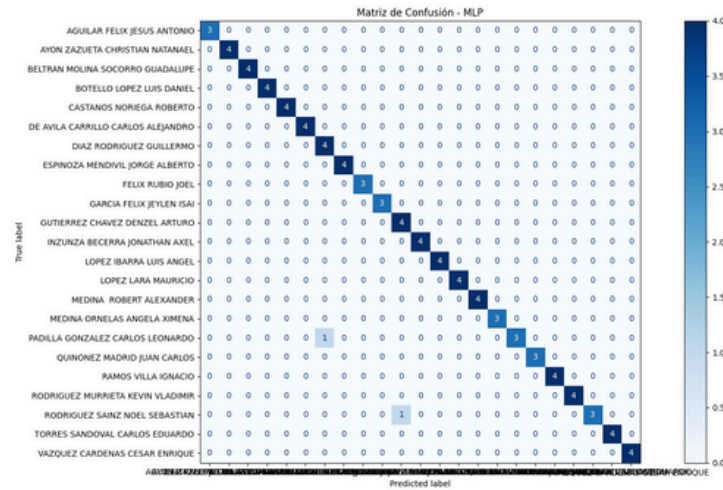
Identificación en Tiempo Real (3_identificar_en_vivo.py)

- Carga de Modelos: Se cargan modelo_svm.joblib (o MLP) y label_encoder.joblib.
- Grabación: Se utiliza sounddevice para grabar un audio de 6 segundos desde el micrófono predeterminado.
- Procesamiento en Vivo: El audio grabado pasa por el mismo proceso de la Fase 1: se convierte a espectrograma Mel y luego a un embedding.
- Predicción: El embedding se pasa al método .predict_proba() del modelo cargado.
- Resultado: El modelo devuelve la clase (número) con mayor probabilidad. Usando el label_encoder cargado, se decodifica el número al nombre de la persona, mostrando la identidad y el porcentaje de confianza.

```
1 def record_and_identify(duration=6, sr=16000):
2     # ... (código de grabación) ...
3
4     # 1. Extraer el embedding del audio grabado
5     embedding = extract_embedding_from_array(audio_data)
6
7     # 2. Predecir con el clasificador
8     prediction_probs = classifier_model.predict_proba(embedding)[0]
9
10    # 3. Obtener la clase (el número) con mayor probabilidad
11    predicted_class_index = np.argmax(prediction_probs)
```

Para evaluar el rendimiento de los clasificadores, se utilizaron los datos de prueba. Las métricas obtenidas.





```

--- Evaluando SVM ---
Modelo guardado en 'modelo_svm.joblib'
Accuracy: 1.0000
F1-Score (weighted): 1.0000

--- Evaluando MLP ---
Modelo guardado en 'modelo_mlp.joblib'
Accuracy: 0.9770
F1-Score (weighted): 0.9766

```

Conclusiones

Aguilar Felix

El proceso central gira en torno a la generación de embeddings, vectores numéricos que representan las características únicas de una voz, utilizando el modelo VoiceEmbedder, que a su vez está basado en Whisper. Este sistema fue estructurado en tres fases: la carga y preprocesamiento de audio para generar y almacenar los embeddings, el entrenamiento de clasificadores, y la identificación en tiempo real. La evaluación del rendimiento de los modelos SVM y MLP demostró una alta eficacia en la tarea de autenticación, con el modelo SVM alcanzando una Precisión (Accuracy) de 1.0000 y un F1-Score (weighted) de 1.0000 en el conjunto de prueba, y el modelo MLP logrando un Accuracy de 0.9770 y un F1-Score de 0.9766. Estos resultados, respaldados por la matriz de confusión, validan la robustez del sistema y el enfoque de usar embeddings para la identificación biométrica de voz.

Vazquez Cardenas

Creamos 3 scripts para poder crear un espectrograma mel como se ve en el primer script y creamos las funciones de los métodos requeridos que eran el SVM Y EL MLP luego en el script 2 entrenamos el modelo con los datos del dataset que se generaron en el primero y dividimos los datos para entrenarlo y tener los datos listos para la hora de la ejecución y ya en el script 3 tenemos un script que se ejecuta con el micrófono principal y se graba un audio de 6 segundos y de eso se crea el espectrograma y se mete a la matriz de confucion para ver quien es el mas probable que sea su voz.

Conclusiones

Medina X

Durante el desarrollo del proyecto de Autenticador de Voz, la implementación de un sistema basado en embeddings generados con VoiceEmbedder, nos permitió representar de forma precisa las características únicas de cada voz. Organizamos el trabajo en tres scripts: el primero para generar el espectrograma Mel y los embeddings, el segundo para entrenar los modelos SVM y MLP con los datos obtenidos, y el tercero para realizar la identificación en tiempo real mediante la grabación de audio desde el micrófono.

Los resultados obtenidos fueron muy satisfactorios, destacando el modelo SVM con un rendimiento excelente, lo que confirma que el sistema es robusto y confiable para la autenticación biométrica por voz. Considero que este proyecto no solo cumplió con los objetivos planteados, sino que también nos permitió comprender mejor la aplicación del aprendizaje automático en sistemas reales de seguridad.